The Five Reached Categories
— and Why Two Could Never Be Reached

Companion to *Do Simulated Conversations Reproduce Real Agent Failures?*

Eduardo Fernandez Salazar
Imperial College London · MSc Artificial Intelligence · Pulpoo / Samsung Polanco

29 July 2026

Abstract

The main comparison report presents all seven reachable failure categories side by side and reports 71% category coverage (5/7). This companion document isolates the other half of that result: it explains, mechanistically, why the two uncovered categories — `comprehension` and `data_gap` — *cannot* appear under the current simulation design at any scale, and it re-presents the real-vs-simulated comparison restricted to the five categories the simulator actually reached, with percentages renormalised over that five-category support. Two results follow. First, the unreached categories are structural zeros, not sampling artefacts: across independent batches of 10 to 1,000 simulated conversations and a pooled corpus of 2,560, both categories occur exactly zero times, and coverage is flat at 71% from N=50 onward. The causes are identifiable design idealisations — over-cooperative LLM personas that rephrase instead of repeating (extinguishing the behavioural signal that defines a comprehension failure), and a fully seeded fake database in which missing-data failures are logically impossible. Second, on the ground where both corpora can compete, the correspondence is substantially better than the headline figure suggests: the Jensen–Shannon divergence between the renormalised five-category failure distributions is **0.099 bits**, versus 0.239 bits on the seven-category support — more than half of the headline divergence is attributable to the two structural zeros rather than to the simulator mis-ranking the failure modes it can express.

1 Purpose and provenance

The main report (`docs/results/real_vs_sim/REPORT.md`) is left unchanged. This document does two things that report deliberately does not:

1. It explains, in depth, why two of the seven reachable categories — `comprehension` and `data_gap` — never appeared in simulation, at any scale, and why this is a structural property of the simulation design rather than bad luck or insufficient sampling.
2. It re-presents the comparison restricted to the five categories the simulator actually reached, with percentages renormalised over that five-category support, so the two corpora are compared like-for-like on the ground where both can actually compete.

All numbers derive from the scorer-parity comparison (`real_vs_sim.json`, taxonomy `1.0-frozen-2026-07-14`, failure threshold `min_score = 3.0`): 1,299 real production conversations (376 failures, 28.9%) versus 390 live-agent simulated conversations (82 failures, 21.0%), both labelled by the *identical* rule-based production scorer, with no LLM in the scoring loop. Scale evidence comes from the scale study (seven independent batches, N = 10 to 1,000, pooled 2,560). Charts are generated by `debugger-platform/reached_only_charts.py`.

2 Quick recap: the seven categories

The frozen taxonomy contains eight production-observable failure categories. One, `delivery_infra` (WhatsApp message-delivery and infrastructure failures, 14 real occurrences), is *unreachable by design*: the simulator talks to the agent over HTTP and has no WhatsApp transport layer, so nothing exists that could fail that way. It is excluded from all coverage claims up front, leaving the seven reachable categories of Table 1.

category	what it means	real failures fla
`loop_stall` (infinite_loop)	agent repeats itself; conversation circles	256 (68
`resolution`	understood but could not resolve; user demanded a human	221 (58
`comprehension`	agent did not understand what the user needed	153 (40
`data_gap`	backend/tool data missing or incomplete	48 (12
`hallucination`	agent fabricated information; customer pushed back	13 (3
`missed_escalation`	frustrated user; agent failed to hand off	11 (2
`silent_abandonment` (premature_exit)	conversation died without closure	9 (2

Table 1: The seven reachable categories. Percentages are of the 376 failed real conversations; categories are multi-label, so they sum to more than 100%.

Of these seven, the simulator reproduced five. The two it never reproduced are not fringe categories: together they account for the first and fourth most common real failure modes, and `comprehension` alone is present in over 40% of real failures. Understanding *why* they never show up matters more than the coverage number itself.

3 The two categories that were never reached

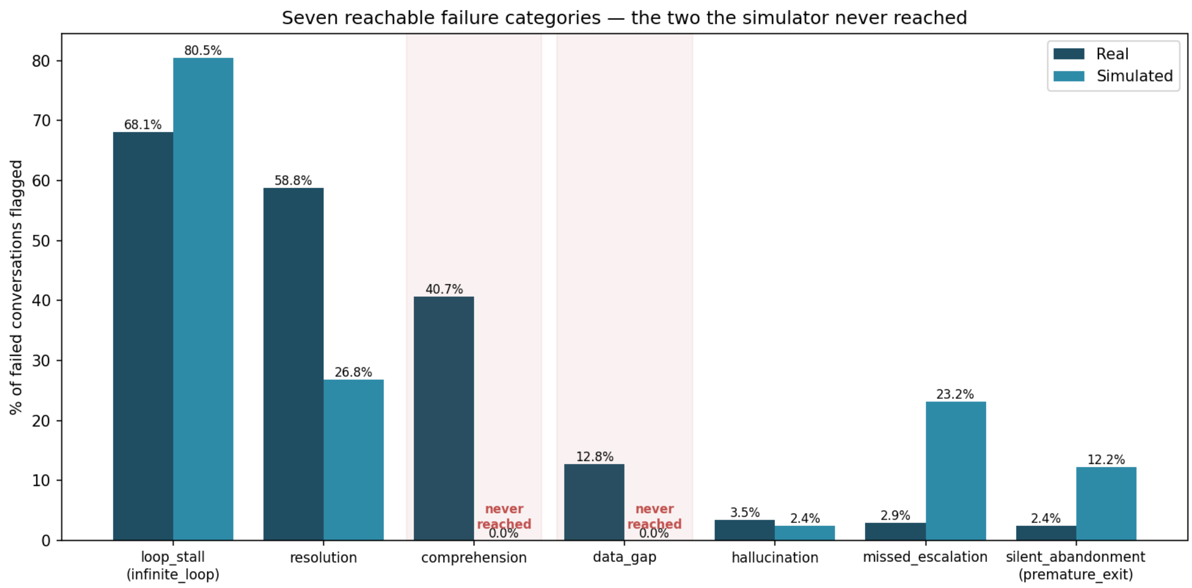

Figure 1: All seven reachable categories, real vs. simulated flag rates among failures. The two categories the simulator never reached are shaded; their simulated bars are exactly zero at every scale tested.

3.1 `comprehension`: the simulated customers are too polite, too clear, and too cooperative

What the category captures in production. A comprehension failure is flagged when the *customer's* behaviour signals that the agent did not understand them — most observably, the customer repeating or re-sending essentially the same request because the agent answered something else. Real WhatsApp customers produce this signal constantly: fragmented messages, slang and regional abbreviations, mixed topics in one message, typos, single-word messages ("precio??"), follow-ups of "?" or "hola?" when ignored — and, crucially, when the agent misunderstands them, they repeat themselves verbatim, often with escalating frustration.

Why the simulator cannot produce it. The simulated customers are themselves LLM personas, and LLM-generated users are systematically *over-cooperative*. When the agent under test gives a confusing or off-target answer, the persona does not do what a real annoyed customer does (paste the same message again, or reply "no, eso no es lo que pregunté"). Instead it behaves like a model trained to be helpful: it *rephrases* its request more clearly, adds context, answers the agent's clarifying questions patiently, and generally repairs the conversation. Grammatically clean, well-structured, single-intent messages are also exactly the inputs the agent under test comprehends best — so persona politeness attacks the failure mode from both sides: it makes misunderstanding less likely to occur, and when it does occur, it erases the behavioural evidence (verbatim repetition) that the scorer needs to flag it. This over-cooperative simulated-user effect is a documented phenomenon in the user-simulation literature (the "Lost in Simulation" line of work), not something unique to this project.

There is a secondary, telemetry-side reason the reachability matrix records this category as only *partially* reachable even in principle: part of the production comprehension signal comes from intent/confidence telemetry emitted by the production stack, which does not exist in the simulation harness. The customer-repeat behavioural signal *is* observable in simulation — it simply never fires, for the reasons above.

In one sentence. Real customers get confused and grind; LLM personas clarify and cooperate, so the behaviour that *defines* a comprehension failure never happens.

3.2 `data_gap`: a fully seeded fake database has no gaps

What the category captures in production. A data-gap failure occurs when the agent does everything right conversationally but the *backend* lets it down: the customer's record is missing, an order is not in the system yet, a field is stale or empty, a lookup returns nothing. The failure originates in the data environment, not in the agent's reasoning. In the real corpus this is 12.8% of failures — real CRMs are messy.

Why the simulator cannot produce it. The simulation environment runs the agent against a single, deliberately *fully seeded* fake customer database, built so that every tool call the agent might make has a complete, well-formed answer — which is exactly what you want when the object under test is the agent's behaviour, and exactly what makes data-gap failures *logically impossible*. Every lookup succeeds; every record exists; every field is populated. There is no incomplete state anywhere in the environment for the agent to stumble over, so no quantity of simulated conversations can surface this category. The failure mode lives in the environment, and the environment was built without it.

In one sentence. You cannot discover missing-data failures in a world where no data is ever missing.

3.3 Why this is structural, not a sampling accident

If the two categories were merely *rare* in simulation, running more simulations would eventually find them. The scale study rules this out. Seven independent batches were run at $N = 10, 50, 100, 200, 400, 800, 1000$, plus a pooled corpus of 2,560 conversations:

- `comprehension` occurrences in simulation: **0 at every N, and 0 in the pooled 2,560**;
- `data_gap` occurrences in simulation: **0 at every N, and 0 in the pooled 2,560**;
- category coverage sits at exactly 5/7 (71%) from N=50 through N=1000 — the curve saturates almost immediately and never moves again. Scaling grows failure *volume* linearly but adds no new failure *kinds*.

A 100× increase in simulation budget changed nothing, which is the empirical signature of a structural ceiling: the generating process for these two categories is *absent*, not undersampled. Buying more of the same simulations buys zero additional coverage; reaching these categories requires *different* simulations (Section 6).

4 The five reached categories, renormalised

4.1 How to read the percentages

The main report expresses each category as a share of *failed conversations* (multi-label: one failed conversation can carry several category flags, so columns sum to more than 100%). That framing is kept in Figure 2. For the composition comparison, the percentages are **renormalised over the five-category support**: each category's flag count is divided by the total number of flags across the five reached categories (real: 510 flags across 376 failures; simulated: 119 flags across 82 failures). This answers the question the raw rates cannot: *given only the failure kinds both worlds can express, how does the failure mass distribute — and does simulation distribute it the way reality does?* Zeroed-out categories no longer distort the comparison, and the two corpora are judged only on ground where both can compete. The renormalised shares describe the composition of *flags*, not of conversations.

	real		simulated		5-category share	
category	count	% of failures	count	% of failures	real	sim
`loop_stall`	256	68.1 (63.2–72.6)	66	80.5 (70.6–87.6)	**50.2%**	**55.5%**
`resolution`	221	58.8 (53.7–63.6)	22	26.8 (18.4–37.3)	**43.3%**	**18.5%**
`hallucination`	13	3.5 (2.0–5.8)	2	2.4 (0.7–8.5)	**2.5%**	**1.7%**
`missed_escalation`	11	2.9 (1.6–5.2)	19	23.2 (15.4–33.4)	**2.2%**	**16.0%**
`silent_abandonment`	9	2.4 (1.3–4.5)	10	12.2 (6.8–21.0)	**1.8%**	**8.4%**

Table 2: The five reached categories. Parenthesised ranges are Wilson 95% confidence intervals on the rate among failures. The final two columns renormalise flag counts over the five-category support (real: 510 flags; simulated: 119 flags).

4.2 Flag rates among failures — the direct comparison

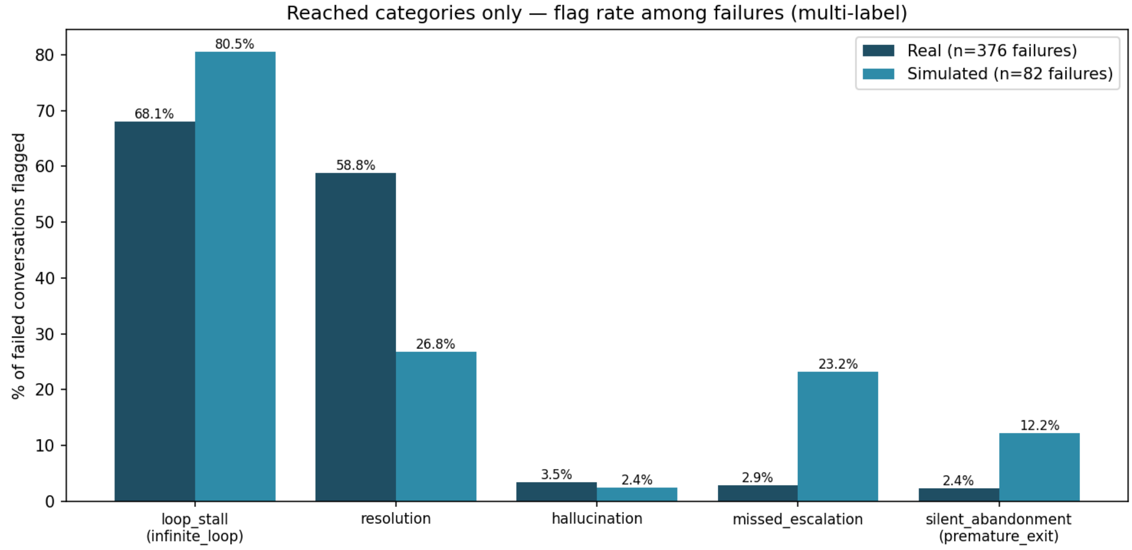

Figure 2: Flag rate among failed conversations for the five reached categories (multi-label). Real: n=376 failures; simulated: n=82.

Reading Figure 2 category by category:

`loop_stall` — reproduced, dominant in both worlds (68.1% vs. 80.5%). The strongest correspondence result in the study: the single most common way the real agent fails — getting stuck repeating itself — is also the single most common way it fails under simulation, at comparable magnitude. Loop behaviour is a property of the *agent* (its retry logic, its inability to break out of a clarification cycle), so it transfers into any environment that lets the conversation run long enough. The simulator slightly over-expresses it, plausibly because personas dutifully keep responding where real customers would have quit, giving loops more room to manifest.

`resolution` — reproduced but substantially under-produced (58.8% vs. 26.8%). Resolution failures need a customer who insists on an outcome the agent cannot deliver and escalates the demand for a human. The same persona politeness that eliminates comprehension failures dampens this one: simulated customers accept partial answers and let the agent off the hook, where real customers keep pushing until the failure is undeniable. The category survives in simulation only because some scenario scripts explicitly direct the persona to demand things the agent cannot do.

`hallucination` — reproduced at close to the real rate (3.5% vs. 2.4%). A small category in both worlds, and the simulated rate lands within sampling noise of the real one. Hallucination is agent-internal — it does not depend on the customer being realistic — which is exactly why it transfers cleanly.

`missed_escalation` — reproduced but heavily over-produced (2.9% vs. 23.2%, $\sim 8\times$). The mirror image of the comprehension story: several test scenarios deliberately script frustrated personas who demand a human, precisely to probe the escalation path. The simulator therefore concentrates escalation pressure far above its natural production frequency. This is over-representation by test design, not a discovery error — but it means the simulated rate measures how the agent handles *engineered* escalation pressure, not how often that pressure arises organically.

`silent_abandonment` — reproduced but over-produced (2.4% vs. 12.2%, $\sim 5\times$). Simulated conversations have hard turn budgets and personas with finite scripted patience, so conversations end without closure more often than in production, where customer abandonment is the tail event.

The overall pattern is coherent rather than random: **agent-internal failure modes (loops, hallucination) transfer at close to their real magnitudes; customer-driven failure modes are distorted in whichever direction the persona design pushes them** — suppressed where personas are too polite (resolution, and comprehension to the point of extinction), inflated where scenarios script the pressure in (escalation, abandonment).

4.3 Composition over the five-category support

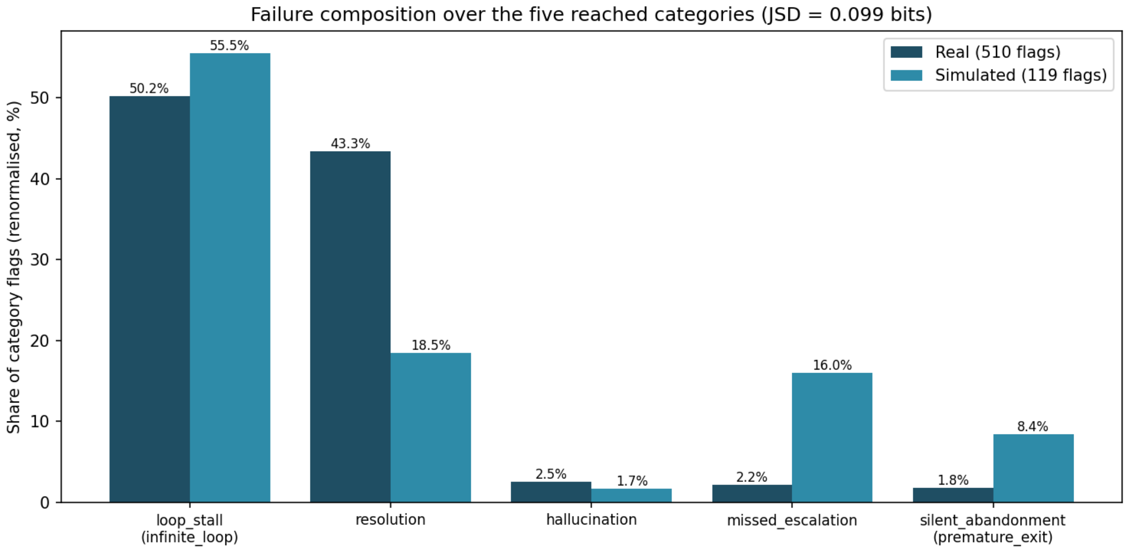

Figure 3: Failure composition renormalised over the five reached categories. JSD = 0.099 bits.

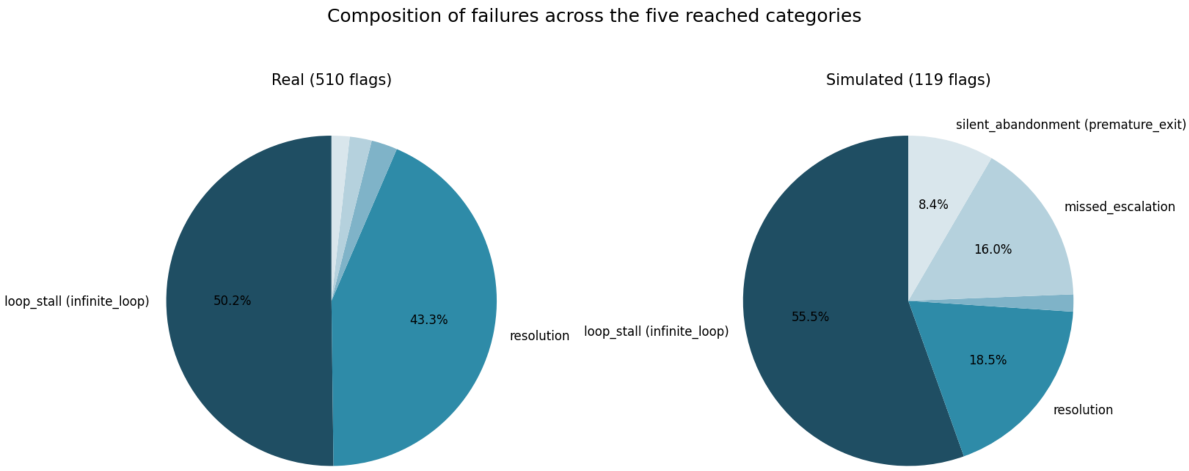

Figure 4: The same renormalised composition as side-by-side pies. Real failure mass is essentially two-mode (loops + resolution $= 93.5\%$); simulation reproduces the loop mode at nearly the same share but redistributes resolution mass into the scripted-pressure categories.

Renormalised, the real failure mass is essentially a *two-mode distribution*: loops (50.2%) plus resolution (43.3%) account for 93.5% of all real flags on this support, with hallucination, escalation, and abandonment sharing the remaining 6.5%. The simulated distribution reproduces the headline structure — `loop_stall` is the plurality mode in both, at nearly the same share (50.2% real vs. 55.5% simulated) — but redistributes the rest: resolution's share drops from 43.3% to 18.5%, and the freed mass flows into the two scripted-pressure categories, `missed_escalation` (2.2% → 16.0%) and `silent_abandonment` (1.8% → 8.4%).

Quantitatively, the Jensen–Shannon divergence between the two renormalised five-category distributions is **0.099 bits** — versus **0.239 bits** on the seven-category reachable support used in the main report (real split-half noise floor: 0.005). More than half of the headline distributional divergence is therefore attributable to the two structurally-zeroed categories, not to the simulator mis-ranking the failure modes it can actually express. On the ground where both corpora can compete, the simulated failure distribution is much closer to reality than the full-support number suggests — though still an order of magnitude above the noise floor, driven by the resolution-deficit / escalation-surplus swap described above.

5 What this means for the overall claim

The fair summary is neither "the simulator found 71% of failure categories" alone, nor "the simulator missed the most common failure mode" alone — it is both, with the mechanism attached:

1. **What simulation is good at:** discovering and reproducing failures that live *inside the agent* — loops, inability to resolve, hallucination, escalation handling, premature termination. These it finds at N as small as 50, with the dominant real mode (`loop_stall`) correctly identified as the dominant simulated mode, and with a five-support composition divergence (0.099 bits) a fraction of the full-support figure.
2. **What this simulation design is blind to:** failures that require the *customer* to be realistically difficult (`comprehension`) or the *environment* to be realistically broken (`data_gap`). These are exactly the ingredients the current design idealises away — cooperative LLM personas and a fully seeded database — and no quantity of additional simulation compensates, as the flat 71% coverage curve from N=50 to N=2560 demonstrates.

The blindness is therefore *diagnosable and, in principle, fixable* — it points at specific components (persona realism, environment seeding), not at simulation as a method.

6 What it would take to reach them (future work)

`comprehension`: persona-realism interventions — frustrated/impatient persona variants that repeat verbatim instead of rephrasing; typo/slang/code-switching noise injection into persona messages; fragmented multi-message inputs; explicit "grinding" behaviour rules (if the agent's reply does not address the request, resend it). Each attacks the over-cooperativeness directly and would let the existing customer-repeat detector fire.

`data_gap`: environment-side fault injection — seed variants of the fake customer database with deliberately missing records, empty fields, and stale values, so agent tool calls can return the incomplete results that define the category. No new detector is required; the failure becomes observable the moment the environment permits it.

`delivery_infra` (outside the seven; unreachable by design): would require simulating transport-level faults (dropped/duplicated/delayed messages), a harness feature rather than a persona or data feature.

7 Caveats

- The renormalised shares are shares of *multi-label flags*, not of conversations; a conversation flagged for both loop and resolution contributes to both categories.
- The simulated failure sample is small (82 failures, 119 flags): simulated-side Wilson intervals are wide, and small-category shares (hallucination especially, n=2) are fragile.
- Both corpora are scored by the same rule-based scorer (scorer parity by construction, no LLM in the loop); human annotation of a sample — and the resulting agreement figure — is still pending, so all category counts inherit the scorer's precision.
- The JSD figures on different supports (0.099 on five categories vs. 0.239 on seven) are not directly comparable in a formal sense — the support differs — but the contrast is precisely the point: it isolates how much divergence the structural zeros contribute.

Charts and numbers: `docs/new_coverage/` (source of record: `docs/results/real_vs_sim/reached_only/`); regenerate with `cd debugger-platforn && python3 reached_only_charts.py`. This document does not modify the main report.